HAVING

Post-Aggregation Group Sieve

SQL FOUNDRY // ARCHITECTURE MANUAL

Phase 4 (Evaluated immediately after GROUP BY)

Complexity: O(K) where K is the number of generated groups.

1. ANSI Syntax & Grammar Specification

```
HAVING aggregate_boolean_expression
    [ AND | OR [NOT] aggregate_boolean_expression ... ]
```

The HAVING clause filters aggregated groups generated by GROUP BY. While WHERE filters individual input tuples before grouping (Phase 2), HAVING evaluates aggregate expressions (e.g., COUNT(*) >= 5, AVG(salary) > 80000) to discard entire groups from the final result set.

2. Database Engine Internals & Storage Footprint

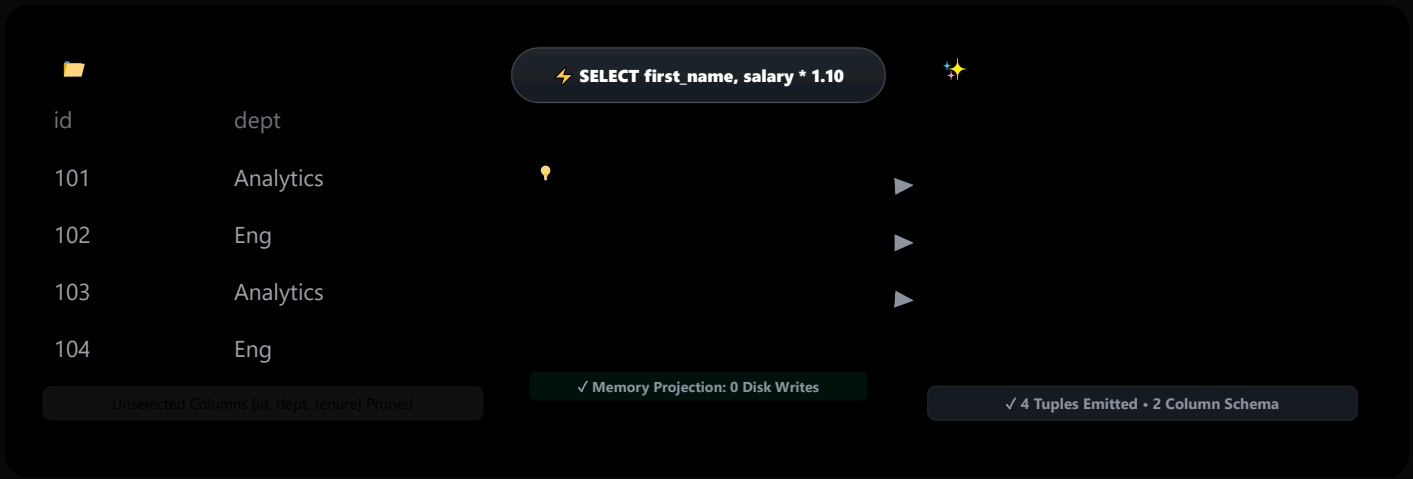
Physical Operator: Post-Aggregate Vector Filter

Memory Footprint: O(1) register evaluation per unique group.

Disk Spill Risk: None. Operates directly on the completed aggregation hash table.

Index SARGability: Cannot directly use indexes because it evaluates computed aggregates. However, pushdown optimizers can rewrite certain non-aggregate HAVING conditions into WHERE filters to leverage indexes before grouping.

3. Relational Architecture Diagram (Tuple Lifecycle)



Overview | 1. Scan Table | 2. Relational Engine | 3. Emit Result

HAVING // IMPLEMENTATION PATTERNS

MODULE: AGGREGATION & ROLLUPS

PAGE 2 OF 4

4. Canonical Query Code Progression

Pattern A: Filtering Groups by Aggregate Headcount

```
-- Retain only departments with 3 or more team members
SELECT
    department,
    COUNT(*) AS team_size,
    AVG(salary) AS avg_comp
FROM enterprise_employees
GROUP BY department
HAVING COUNT(*) >= 3;
```

Pattern B: Multi-Condition Business Outlier Detection

```sql
-- Identifying merchants with high chargeback ratios and volume
SELECT
    merchant_id,
    COUNT(*) AS total_transactions,
    SUM(amount) AS total_volume_usd,
    SUM(CASE WHEN is_disputed = 1 THEN 1 ELSE 0 END) AS disputed_count
FROM merchant_charge_blotter
GROUP BY merchant_id
HAVING COUNT(*) >= 100
   AND SUM(CASE WHEN is_disputed = 1 THEN 1 ELSE 0 END) * 1.0 / COUNT(*) > 0.02;
```

Pattern C: HAVING without GROUP BY (Table-Level Invariant Check)

```sql
-- Check if the entire ledger violates net variance invariant
SELECT
    SUM(debit_amount) AS total_debits,
    SUM(credit_amount) AS total_credits,
    ABS(SUM(debit_amount) - SUM(credit_amount)) AS net_imbalance
FROM general_ledger_postings
HAVING ABS(SUM(debit_amount) - SUM(credit_amount)) > 0.001;
```

5. Production Schemas & Input/Output Relational Tables

Input: 2 Formed Department Groups

department	count	avg_salary
Engineering	3	$102,333.33
Analytics	2	$105,000.00

Emitted Output Table

department	count	avg_salary
Engineering	3	$102,333.33

6. Production Enterprise Case Study

Company / Team: JPMorgan Chase Treasury & General Ledger Accounting

Operational Scenario: Automated end-of-day reconciliation sweeps scan 850 million debit and credit journal entries across global clearing entities. An alert must fire if any trading entity has a non-zero ledger variance.

```sql
SELECT
    legal_entity_id,
    currency_code,
    SUM(debit_amount_usd) AS total_debits,
    SUM(credit_amount_usd) AS total_credits,
    ROUND(SUM(debit_amount_usd) - SUM(credit_amount_usd), 4) AS ledger_variance
FROM treasury_journal_entries
WHERE posting_date = CURRENT_DATE
GROUP BY legal_entity_id, currency_code
HAVING ABS(SUM(debit_amount_usd) - SUM(credit_amount_usd)) > 0.0001;
```

Business & Infrastructure Impact: Instantly identifies out-of-balance legal entities prior to official Federal Reserve EOD regulatory reporting.

7. FAANG & Wall Street Technical Interview Defense

[Citadel / Two Sigma] Question 1:

What happens when you use a HAVING clause WITHOUT a GROUP BY clause?

Model Answer: When HAVING is used without GROUP BY, the entire table is treated as a single synthetic group. If the aggregate condition evaluates to TRUE, exactly one row is emitted; if it evaluates to FALSE, zero rows are emitted. This is frequently used for table-wide invariant checks.

[Stripe] Question 2:

Explain why putting row-level filters in HAVING instead of WHERE degrades database performance.

Model Answer: WHERE filters rows at Phase 2, preventing non-matching rows from ever entering the memory hash table. If you put non-aggregate conditions in HAVING, the engine must ingest, hash, and aggregate millions of useless rows only to discard them at Phase 4, increasing memory usage and query time by 10x-100x.

---

8. High-Yield Pitfalls & Corporate Production Bugs

High **Filtering Raw Columns in HAVING:** Writing "HAVING status = 'ACTIVE'" forces the engine to aggregate inactive rows before filtering. Always move non-aggregate predicates into WHERE.

Critical **Confusing WHERE vs HAVING Placement:** Attempting to use aggregate functions like COUNT(*) > 5 in WHERE triggers immediate syntax compiler errors.

9. Certification & Conceptual Mastery Examination (5 MCQs)

1. Which of the following expressions is VALID in a HAVING clause but INVALID in a WHERE clause?
A) status = 'PAID'
B) created_at >= '2026-01-01'
C) COUNT(order_id) > 10
D) customer_id IS NOT NULL
Correct Answer: C • *COUNT() is an aggregate function that requires grouping to evaluate. WHERE evaluates before grouping occurs.*

2. If a query has both WHERE and HAVING, in what order are they evaluated?
A) HAVING evaluates first, then WHERE
B) Simultaneously
C) WHERE evaluates at Phase 2 (row-level); HAVING evaluates at Phase 4 (group-level)
D) The database optimizer randomly chooses
Correct Answer: C • *WHERE always filters candidate rows prior to GROUP BY. HAVING evaluates aggregated group metrics after grouping.*

3. Can you reference SELECT column aliases inside the HAVING clause in standard ANSI SQL?
A) Yes, always
B) No, because HAVING evaluates at Phase 4, before SELECT assigns aliases at Phase 6/7
C) Only in Oracle and DB2
D) Only if the alias is uppercase
Correct Answer: B • *In strict ANSI SQL, HAVING evaluates before SELECT. (Some dialects like Snowflake and MySQL offer non-standard extensions, but standard ANSI requires repeating the aggregate expression).*

4. What is the output of "SELECT COUNT(*) FROM employees HAVING COUNT(*) > 100" if the table has only 50 rows?
A) 50
B) 0 rows (Empty result set)
C) NULL
D) Runtime error

Correct Answer: B • *The single table-level group has a COUNT(*) of 50. Since 50 > 100 is FALSE, the HAVING filter discards the entire group, returning an empty set.*

5. Why is "WHERE dept = 'Sales' GROUP BY dept" preferred over "GROUP BY dept HAVING dept = 'Sales'"?

A) They are identical in performance

B) WHERE allows index usage and avoids building hash buckets for non-Sales departments

C) HAVING does not work with strings

D) WHERE requires less typing

Correct Answer: B • *WHERE eliminates rows before aggregation, allowing B-Tree index scans and keeping the aggregation hash table small.*